

## Section 01

# Industrial Security (Master)



## Section 01 – Firmware Reverse Engineering

### Firmware Reverse Engineering

*Lecture Notes*

Department of Computer Science

- 1 Why Firmware Matters
- 2 Sourcing and Triage
- 3 Deep Analysis
- 4 Responsible Disclosure

## By the end of this section you will be able to

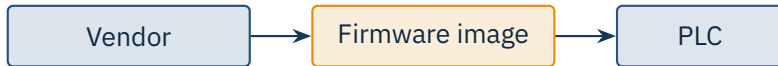
- Source firmware images lawfully and verify their integrity.
- Identify the architecture, bootloader, and root filesystem inside an image.
- Use binwalk, firmware-mod-kit, and Ghidra to peel a firmware.
- Recognise common backdoors and hard-coded secrets.
- Explain the responsible-disclosure path for ICS findings.

# 1

---

## Why Firmware Matters

Section 01 – Firmware Reverse Engineering



## The opaque box

Without firmware analysis the asset owner only sees what the vendor chooses to document. Backdoors, hard-coded credentials, and weak crypto often hide here.

- **Project Basecamp (2012)** – 16 PLC families with hard-coded credentials.
- **Schneider M340 (2018)** – Modicon firmware downgrade without signing.
- **Siemens S7-1200 (2019)** – engineering-tool backdoor for bootloader updates.
- **CODESYS Runtime (2023)** – *Code16* suite, 16 CVEs affecting hundreds of products.
- **Boa web server** – abandoned upstream, still in many ICS HMIs (Microsoft, 2022).

## Pattern

Long product lifecycles + slow vendor patching + shared third-party code = systemic risk.

# 2

---

## Sourcing and Triage

Section 01 – Firmware Reverse Engineering

- Vendor support portal (per warranty / contract).
- Downloaded from a device you own (read-out tools, JTAG).
- Public update CDN (URLs often discoverable via cached references).
- Manufacturer's open partner programme (Siemens Industry Online, Rockwell PCDC).

## Caution

Compliance Reverse engineering is often legally permitted for security research and interoperability. Check your jurisdiction; in the EU see Directive 2009/24/EC and the CRA's research safe-harbour.



```
1 $ binwalk -e firmware.bin
2 DECIMAL      HEX          DESCRIPTION
3 0            0x0         uImage header, header size: 64 bytes, ...
4 64          0x40        LZMA compressed data, ...
5 1572864      0x180000    Squashfs filesystem, little endian, ...
```

## Signals

File magic at well-known offsets, recognisable compression algorithms, sane image checksum – and you have probably found bootloader, kernel, and root filesystem.



## Cheap wins first

Before opening Ghidra, run `strings`, `grep -i pass|key|token`, and look at `/etc/passwd`. Many findings sit on the surface.

# 3

---

## Deep Analysis

Section 01 – Firmware Reverse Engineering

ARM (Cortex-M / A)

MIPS (legacy)

PowerPC

Renesas RX/RH

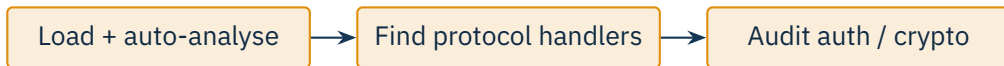
## Detection tools

`file`, `binwalk -A`, `cpu_rec`, and the Ghidra “Auto Analyze” loader. Cross-check against the chip on the actual board if you can.

```
1 $ strings -n 8 -a firmware.bin | grep -iE 'password|key|secret|admin'  
2 admin  
3 Admin123  
4 SerialPort_default_key=4f6b5e...  
5 $ binwalk -E firmware.bin      # high-entropy regions often hide keys
```

## Red flags

Strings like “admin” adjacent to “pass...”, high-entropy 16/32-byte blobs near init code, repeated keys across firmware versions.



## Workflow tip

For repeat work, save your Ghidra project per vendor family. Common findings (Modbus parser, web admin, OTA verifier) tend to recur across products.

# 4

---

## Responsible Disclosure

Section 01 – Firmware Reverse Engineering



## Timelines

Common practice: 90 days vendor + 30 day grace. ICS vendors often need longer (180 days) because of co-ordinated firmware rollouts.



- Do not test on production hardware unless it is yours.
- Do not weaponise findings before vendor disclosure is complete.
- Keep proof-of-concept code in a private repo until co-ordinated release.
- Track legal cover (DMCA §1201 in the US, EU CRA research safe-harbour, BSI §202c in Germany).

## Caution

Hard line A finding obtained on equipment you do not own (or are not authorised to test) is contaminated – no responsible publication channel will accept it.

### ★ Key Takeaway: Firmware RE for ICS in one breath

- Source the image lawfully, verify integrity, identify the architecture.
- binwalk + strings answer 80% of triage questions for free.
- Open Ghidra only after the cheap signals are exhausted.
- Hard-coded credentials and unauthenticated firmware-update paths are still common findings in 2020s ICS devices.
- Coordinated disclosure (vendor + national CERT) is the only ethical publication route.

# End of Section 01



Firmware Reverse Engineering

Questions and discussion